Bernoulli Naive Bayes

it is a variation or subcategory of the Naive Bayes algorithm.
it is typically used when the data is binary and it models the occurrence of features using Bernoulli distribution.
it is used for classification of binary feature such as "yes" or "no" or "0" or "1", "true" or "false"...

It assumes that each feature is conditionally independent given the class y. so we can calculate the likelihood of each feature occurring as:

$$p(x_i|y) = p(i|y)x_i + (1 - p(i|y))(1 - x_i)$$

- $p(x_i|y)$ is the conditional probability of x_i occurring provided y has occurred
- i is the event
- x_i holds binary value either 0 or 1

Bernoulli Distribution

$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$

$$f(x) = \begin{cases} p^x * (1 - p)^{1-x} & \text{if x=0,1} \\ 0 \; otherwise \end{cases}$$

Example:

To understand how Bernoulli Naive Bayes works, here's a simple binary classification problem.

Message ID	Message Text	Class
M1	"buy cheap now"	Spam
M2	"limited offer buy"	Spam
M3	"meet me now"	Not Spam
M4	"let's catch up"	Not Spam

1. Vocabulary

Extract all unique words from the training data:

$$\text{Vocabulary} = \{\text{buy, cheap, now, limited, offer, meet, me, let's, catch, up}\}$$

Vocabulary size $V = 10$

2. Binary Feature Matrix (Presence = 1, Absence = 0)

Each message is represented using binary features indicating the presence (1) or absence (0) of a word.

ID	buy	cheap	now	limited	offer	meet	me	let's	catch	up	Class
M1	1	1	1	0	0	0	0	0	0	0	Spam
M2	1	0	0	1	1	0	0	0	0	0	Spam
M3	0	0	1	0	0	1	1	0	0	0	Not Spam
M4	0	0	0	0	0	0	0	1	1	1	Not Spam

3. Apply Laplace Smoothing

$$P(w_i = 1 \mid C) = \frac{\text{count}(w_i, C) + 1}{N_C + 2}$$

where $N_C = 2$ for both classes (2 documents per class), so the denominator becomes 4.

4. Word Probabilities

For Spam class:

- $P(\text{buy} \mid \text{Spam}) = \frac{2+1}{4} = 0.75$
- $P(\text{cheap} \mid \text{Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{now} \mid \text{Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{limited} \mid \text{Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{offer} \mid \text{Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{others} \mid \text{Spam}) = \frac{0+1}{4} = 0.25$

For Not Spam class:

- $P(\text{now} \mid \text{Not Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{meet} \mid \text{Not Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{me} \mid \text{Not Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{let's} \mid \text{Not Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{catch} \mid \text{Not Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{up} \mid \text{Not Spam}) = \frac{1+1}{4} = 0.5$
- $P(\text{others} \mid \text{Not Spam}) = \frac{0+1}{4} = 0.25$

5. Classify Message "buy now"

The message contains words "buy" and "now, so the feature vector is:

$$\text{buy} = 1, \quad \text{now} = 1, \quad \text{others} = 0$$

5.1 For Spam:

- $P(\text{Spam} \mid d) \propto P(\text{Spam}) \cdot P(\text{buy} = 1 \mid \text{Spam}) \cdot P(\text{now} = 1 \mid \text{Spam}) = 0.5 \cdot 0.75 \cdot 0.5 = 0.1875$

5.2 For Not Spam:

- $P(\text{Not Spam} \mid d) \propto P(\text{Not Spam}) \cdot P(\text{buy} = 1 \mid \text{Not Spam}) \cdot P(\text{now} = 1 \mid \text{Not Spam}) = 0.5 \cdot 0.25 \cdot 0.5 = 0.0625$

6. Final Classification

$$P(\text{Spam} \mid d) = 0.1875, \quad P(\text{Not Spam} \mid d) = 0.0625$$

Since $P(\text{Spam} \mid d) > P(\text{Not Spam} \mid d)$, the message is classified as: $\boxed{\text{Spam}}$

Code example from GeekforGeek

```
import numpy as np
import pandas as pd
from sklearn.naive_bayes import BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer

df=pd.read_csv("spam_ham_dataset.csv")
print(df.shape)
print(df.columns)
df= df.drop(['Unnamed: 0'], axis=1)

x = df["text"].values
y = df["label_num"].values

cv = CountVectorizer()

x = cv.fit_transform(x)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.20,
random_state=0)\

bnb = BernoulliNB(binarize=0.0)
model = bnb.fit(X_train, y_train)
y_pred = bnb.predict(X_test)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.84      0.98      0.91       732
           1       0.92      0.56      0.70       303

    accuracy                           0.86      1035
   macro avg       0.88      0.77      0.80      1035
weighted avg       0.87      0.86      0.84      1035
```

from Geek4Geek